Fully Automated HTML and Javascript Rewriting for Constructing a Self-healing Web Proxy

Thomas Durieux
Inria & University of Lille
Lille, France

Youssef Hamadi
Ecole Polytechnique
Paris, France

Martin Monperrus
KTH Royal Institute of Technology
Stockholm, Sweden

arXiv:1803.08725v1 [cs.SE] 23 Mar 2018

ABSTRACT

Over the last few years, the complexity of web applications has increased to provide more dynamic web applications to users. The drawback of this complexity is the growing number of errors in the front-end applications. In this paper, we present BikiniProxy, a novel technique to provide self-healing for the web. BikiniProxy is designed as an HTTP proxy that uses five self-healing strategies to rewrite the buggy HTML and Javascript code. We evaluate BikiniProxy with a new benchmark of 555 reproducible Javascript errors, DeadClick. We create DeadClick by randomly crawling the Internet and collect all web pages that contain Javascript errors. Then, we observe how BikiniProxy heals those errors by collecting and comparing the traces of the original and healed pages. To sum up, BikiniProxy is a novel fully-automated self-healing approach that is specific to the web, evaluated on 555 real Javascript errors, and based on original self-healing rewriting strategies for HTML and Javascript.

1 INTRODUCTION

According to [2], at least 76% of all websites on the Internet use Javascript. The Javascript code used in today's web page is essential: it is used for social media interaction, dynamic user-interface, usage monitoring, advertisement, content recommendation, fingerprinting, etc, all of this being entirely part of the "web experience". For example, when a user browses the website cnn.com, she is loading more 125 than Javascript files, which represent a total of 2.8 megabytes of code.

The drawback of this complexity is the growing number of errors in web pages. For instance, a common Javascript error is due to uninitialized errors, resulting in an error message such as `cannot read property X of null` where `X` is a variable name. Searching for this error message on Github and Google respectively gives 179000 and 27318 results (data from March 9th, 2018). Ocariza at al. [27] have performed a systematic study showing that the majority of the most visited websites in 2011 contain Javascript errors.

In this paper, we propose a novel technique to provide self-healing for the web. It is along the line of previous work on self-healing software (e.g. [16]), also called failure-oblivious computing (eg. [32, 14]), automated recovery and remediation (e.g. [8]). The majority of the self-healing literature focuses on the C/Unix runtime. On the contrary, we are interested in the Javascript/browser runtime, which is arguably much different. Indeed, the topic of self-healing of Javascript errors is a very little researched area [11, 12].

Our novel self-healing technique is founded on two insights. Our first key insight is that a proxy between the client browser and the web server can be used for providing the self-healing capabilities. Our second key insight is that the most common Javascript errors can be fixed by automated rewriting of HTML or Javascript code.

In this paper, we present a novel self-healing proxy for the web, called BikiniProxy. It consists of five self-healing strategies, that are specifically designed for Javascript errors. Those strategies are based rewriting, defined as automated modification of the code: automated addition or modification of HTML elements and automated transformation of Javascript abstract syntax trees.

Our approach does not make any assumption on the architecture or libraries of web applications. First, proxy servers are used in most web architectures. Second, our approach does not require a single change to existing web pages and applications. As such, BikiniProxy is highly applicable and could be embedded in a service like Amazon/CloudFlare[1].

We evaluate our approach as follows. First, we set up a crawler to randomly browse the Internet, for each browsed page, it logs the Javascript errors, if any, occurring during the loading of the page content. Second, we observe how BikiniProxy heals those errors by collecting and comparing traces. Over 8 full days, our crawler has visited 96174 web pages, and identified 4282 web pages with errors. We observed that 3727 errors were either transient (due to asynchronicity [27]) or fixed by the developer after crawling. Eventually, we evaluated BikiniProxy on 555 web pages with errors, representing an unbiased sample of real field errors. BikiniProxy is able to make all errors disappear in 176/555 of the cases, that is 31.76%. In the best cases, the self-healing provides the user with new features or new content. We provide detailed qualitative and quantitative analysis on the main categories of self-healing outcome. To sum up, BikiniProxy is a novel fully-automated self-healing approach that is specific to the web, evaluated on 555 real Javascript errors, and based on original self-healing rewriting strategies for HTML and Javascript.

Our contributions are:

- The design and implementation of a novel self-healing proxy, BikiniProxy, for today's web.
- Five self-healing strategies for the web, specifically designed to automatically recover from the most frequent errors in Javascript.
- A benchmark of 555 real web pages with Javascript errors. Special care has been taken so that all error are fully reproducible for today and future experiment in this research area.
- An evaluation of BikiniProxy over the 555 Javascript errors from our benchmark, showing that BikiniProxy makes all errors disappear for 31.76% of web pages with errors. This quantitative evaluation is complemented by a qualitative analysis of BikiniProxy's effectiveness.
- An open-source implementation of BikiniProxy, publicly available for future research [1].

[1] https://cloudflare.com

The remainder of this paper is organized as follows. Section 2 explains the background of BikiniProxy. Section 3 details the BikiniProxy approach. Section 4 details the evaluation. Section 5 details the threats to validity of the contribution. Section 6 presents the related works and Section 7 concludes.

2 BACKGROUND

2.1 The Complexity of Today's Web

A web page today is a complex computational object. A modern web page executes code and depends on many different resources. Those resources range from CSS styles of thousands of lines, external fonts, media objects, and last but not least, Javascript code.

For example, when a user browses the website cnn.com, he is loading more than 400 resources and 125 of them are Javascript, which represents a total of 2.8mb of code. Anecdotally, back in 2010, the same web page cnn.com contained 890kb of Javascript code [27] (68% less code!)

Today's web page Javascript is essential: it is used for social media interaction, usage monitoring, advertisement, content recommendation, fingerprinting, etc, all of this being entirely part of the "web experience". Consequently, 76% of all websites on the Internet use Javascript [2]. To this extent, a web page today is a program, and as such, suffers from errors.

2.2 Javascript Errors

Web pages and applications load and execute a lot of Javascript code [25]. This code can be buggy, in fact, the top 100 of the most visited websites contains Javascript errors [27]. One kind of Javascript error is an uncaught exception, which is similar to uncaught exceptions in modern runtimes (Java, C#, Python). Those errors are thrown during execution if the browser state is invalid like when accessing a property on a null element (a null dereference).

If an error is not caught by the developer, the execution of the current script is stopped. In Javascript, there is a different "execution scope" for each loaded scripts (i.e. for each HTML script tag) and for each asynchronous call. Consequently, contrary to classical sequential execution, in a browser, only the current execution scope is stopped, and the main thread continues running. This means that one can observe several uncaught exceptions for a single page.

The uncaught errors are logged in the browser console that is accessible with the developer tool. Most browsers provide an API to access all the errors that are logged in the browser console. This means that it is relatively easy to monitor Javascript errors in web applications.

2.3 Web Proxies

A web proxy is to be an intermediate component between a client browser and web server. In essence, a proxy transmits the client request to the server, receives the response from the server, and forwards it to the client browser. On the web, proxies are massively used for different purposes.

(1) A `Network Proxy` is used to expose a service that is not directly accessible, because of network restrictions [22]. The proxy, in this case, is a bridge between two networks and its only task is to redirect requests and responses. For example, a popular network proxy is nginx, it is used to expose websites on port 80 which are indeed served on other ports > 1024, this avoids granting root access to web applications.
(2) A `Cache Proxy` is a proxy that is used to cache some resources directly in the proxy in order to improve the serving time to an external resource [29]. The cache proxy stores the response of the server locally and if a request is made for the same resource, the local version is directly sent to the client without sending the request to the server. A wildly used cache proxy is, for example, a content-delivery network (CDN) that provides optimized access to static resources on the Internet.
(3) A `Security Proxy` is used to verify whether a client browser is legitimate to access a server [19]. This type of proxy can be used for example to protect a server against Denial-of-service attacks.
(4) A `Load-balancer Proxy` is used on popular applications to distribute the load of users on different backend servers [7]. A load balancer can be as simple as a round-robin, but can also be more sophisticated. For instance, a load-balancer can try to find the least loaded server available in the pool.

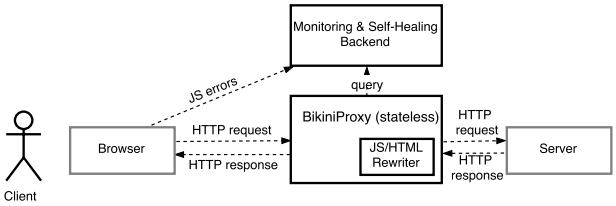


Figure 1: The Architecture of BikiniProxy. The key idea is that all requests are proxied by "BikiniProxy". Then, if an error is detected, a self-healing strategy based on HTML and/or JS rewriting is automatically applied.

2.4 Self-healing Software

Self-healing software refers to the idea that it is possible to engineer fully automated error recovery for software. Berger nicely puts it as software seatbelts and airbags [6]. For instance, self-healing can consist of restoring invariants [13]. The literature uses different expressions to refer to this very same idea: automated recovery [15], failure-oblivious computing [32], runtime repair [20]. Note that "runtime repair" and "program repair", although they share the word "repair" are fundamentally different [23, 5]: the former happens at runtime in production, and does not involve patches for developers, while the latter is offline and consists of generating patches for human developers. Despite its potential impact, self-healing capabilities for web applications is a very little researched area [11, 12].

In this paper, we propose a novel kind of web proxy, a self-healing proxy, for automatically recovering at runtime from Javascript errors in web browsers.

3 CONTRIBUTION

We now present BikiniProxy, a novel approach to fully automate self-healing of HTML and Javascript in production.

The intuition behind BikiniProxy is that a proxy between web applications and the end-users could provide the required monitoring and intercession interface for automatic error handling. This is the concept of ‘‘self-healing proxy’’ that we explore in this paper.

3.1 Architecture

Figure 1 presents the architecture of BikiniProxy. BikiniProxy is composed of three main parts.

(1) BikiniProxy (see Section 3.1.1) is a stateless HTTP proxy that intercepts the HTML and Javascript requests between the browser (also called “client” in this paper) and the web server.
(2) The JS/HTML Rewriter service (see Section 3.1.2) that contains the self-healing strategies to handle Javascript errors.
(3) The Monitoring and Self-healing Backend (see Section 3.1.3) stores information about the known errors that have happened and the success statistic of each self-healing strategies for each error.

Let us start with a concrete example. Bob, a user of the website http://foo.com browses the page `gallery.html` and uses BikiniProxy to improve his web experience. Since BikiniProxy is a proxy, when Bob opens `gallery.html`, the request goes through the proxy. When the request is made, BikiniProxy queries the backend to know whether another, say Alice, has experienced errors on `gallery.html`. Indeed, Alice's browser got a `jQuery is not defined` error two days before. The backend sends this error to the proxy, which consequently launches the JS/HTML Rewriter service to handle the error. For `gallery.html`, the rewriting is HTML-based and consists of by injecting the library jQuery in the HTML response. BikiniProxy also injects its error monitoring framework before sending the rewritten response to Bob's browser. The rewritten page is executed by Bob's browser, BikiniProxy's monitoring tells the proxy that Alice's error does not appear anymore, meaning that the rewriting strategy made it disappeared.

Algorithm 1 shows the workflow of BikiniProxy. BikiniProxy receives the HTTP request from the browser (line 1). Then it redirects the request to the Web Server (line 2) like any proxy. For each HTML response, BikiniProxy injects a framework (line 4) to monitors the Javascript errors in the client browser. When an error happens on the client browser, it is sent to BikiniProxy's backend for being saved in a database.

BikiniProxy uses the backend to know which Javascript resource has thrown an error in the past: for each HTML and Javascript resource, BikiniProxy queries the backend service with the URL of the requested resource to list all the known errors (line 6). If there is at last one known error, BikiniProxy triggers the JS/HTML Rewriter service to apply the self-healing strategies on the fly to the requested content (line 10). Then the response is sent to the client (line 15) with a unique id to monitor the effectiveness of the applied self-healing strategy.

Definition 3.1 (Resource). A web **resource** is a content on which a web page is dependent. For instance, an image or a Javascript script is a web resource. In this paper, a web resource is defined by 1) an URL to address the resource; 2) its content (whether text-based or binary-based) and 3) the HTTP headers that are used to serve the resource. The resource can be used as an attribute of an HTML tag (<script>, <img>, <link >, <iframe>, etc.) or used as an AJAX content.[2]

Algorithm 1 The main BikiniProxy algorithm

Input: B: the client browser
Input: W: the Web Server
Input: R: the rewriting services
Input: D: BikiniProxy Backend
1: **while** new HTTP *request* from B **do**
2: $response \leftarrow W(request)$
3: **if** request is html page **then**
4: $response \leftarrow inject_bikiniproxy_code(response)$
5: **end if**
6: $errors \leftarrow D.previous_errors_from(request_{url})$
7: **if** errors is not empty **then**
8: **for** r in R **do**
9: **if** $isToApply(r, errors, request, response)$ **then**
10: $response \leftarrow r.rewrite(response, request, errors)$
11: $response \leftarrow response + uuid$
12: **end if**
13: **end for**
14: **end if**
15: $send(response)$
16: **end while**

3.1.1 The Proxy. A proxy intercepts the HTML code and the Javascript code that is sent by the web server to the client browser. By intercepting this content, the proxy can modify the production code and therefore change the behavior of the web application. One well-known example of such a change is to minimize the HTML and Javascript code to increase the download speed.

In BikiniProxy, the proxy automatically changes the Javascript code of the web application to handle known errors. BikiniProxy is configured with what we call “self-healing strategies”. A self-healing strategy is a way to automatically recover from a certain class of errors, see Section 3.1.2. In BikiniProxy, the proxy is made stateless, in order to scale well to massive amounts of requests.

3.1.2 JS/HTML Rewriter. The role of the JS/HTML Rewriter is to rewrite the content of the Javascript and HTML resources in order to: 1) monitor the Javascript errors that happen in the field 2) change the behavior when a Javascript resource has been involved in an error in the past. In this paper, a “known error” is an error that has been thrown in the browser of a previous client, that has been detected by the monitoring feature of BikiniProxy and that has been saved in the BikiniProxy backend (see Section 3.1.3).

We design for BikiniProxy, five self-healing strategies that target the most frequent Javascript errors that we observe when we craw the Internet.

(1) HTTP/HTTPS Redirector that changes HTTP URLs to HTTPS URLs.
(2) HTML Element Creator that creates missing HTML elements.
(3) Library Injector injects missing libraries in the page.
(4) Line Skipper wraps a statement with an if to prevent invalid object access.

[2] AJAX means requested programmatically in Javascript code

(5) Object Creator initializes a variable with an empty object to prevent further null dereferences.

In addition, the JS/HTML Rewriter is plugin-based, it can be easily extended with new self-healing strategies to follow the fast evolution of the web environment.

HTTP/HTTPS Redirector. The modern browsers have a policy to block unsecured content (i.e. HTTP resources) in secured web pages (HTTPS). For example, `https://foo.com` cannot load `http://foo.com/f.js` (no https). The rationale is that the unsecured requests can be easily intercepted and modified to inject scripts that will steal information from the secure page, for example, your banking information.

As of 2018, we still are in a period of transition where both HTTP and HTTPS web pages exist, and some websites provide access to their content with both HTTP and HTTPS protocol. Consequently, it happens that the developers forget to change some URL in their HTTPS version and those resources are blocked, resulting in incomplete web pages or Javascript errors.

The self-healing strategy of HTTP/HTTPS Redirector is to change all the HTTP URL by HTTPS URL in HTTPS pages. By doing this, all resources are loaded and the unwanted behavior due to blocked resources is fixed.

HTML Element Creator. As shown by Ocariza et al. [25], most of the Javascript errors are related to the DOM. This is especially true when the developers try to dynamically change the content of a specific HTML element using getElmentById, like:

```
document.getElmentById("elementID")
    .innerText = "Dynamic content";
```

Since the HTML and the Javascript are provided in different files, it is not rare that an HTML element with an ID is removed or changed without changing the associated Javascript code. For example, if DOM element `"elementID"` is removed, `document.getElmentById(...)` returns `null` and the execution results on a null dereference when the property `innerText` is set.

When an error happens, BikiniProxy determines if the error is related to the access of a missing element in the DOM: it does so by looking at the Javascript code at the failure point. If it is the case, "HTML Element Creator" extracts the query that the Javascript used to access the element and create an empty and invisible HTML element in the DOM. The Javascript code then runs without error and the execution continues without affecting the client browser.

Library Injector. In Javascript, it is a common practice to rely on external libraries to facilitate the development of the web applications. Some of these libraries are extremely popular and are used by millions of users every day, like jQuery or AngularJS. Sometimes, these libraries are not correctly loaded into web pages, and this produces a very characteristic error: for example, `jQuery is not defined` BikiniProxy parses those errors and determines which library is missing.

To do so, BikiniProxy has an initial offline training phase, missing libraries are simulated on a test website, and reference errors are collected for the top 10 libraries presented in [2]. Based on these reference errors, error parsing rules have been manually written to determine which library is missing. When Library Injector detects that a web page contains an error related to a missing library, it injects the related library in the web page. The rewritten page contains the missing library and the web page can be completely loaded.

Algorithm 2 Algorithm to rewrite JS code with "Line Skipper" strategy

Input: E: error
Input: R: resource
1: (line, column, resource_url) = extractFailurePoint(E, R.body)
2: **if** resource_url is R.url **then**
3: ast = getAST(R.body)
4: elem = getElementFromAst(ast, line, column)
5: wrapElemWithIf(elem)
6: R.body = writeAST(ast)
7: **end if**

Line Skipper. The errors `XXX is not defined` and `XXX is not a function` in Javascript are errors that are related to invalid access to a variable or a property. `XXX is not defined` is triggered when an identifier (name of variable / function) is used but was never defined in the current scope. For example if we call `if(m){}` without defining `m`, the execution ends with the error `'m' is not defined`.

The second error, `XXX is not a function`, is triggered when a variable that is not a function is called. For example, the code `var func = null; func()` will trigger the error `func is not a function`.

To avoid these errors, Line Skipper wraps the statement that contains the invalid code with an if that verifies that the element is correctly defined for the first error, `if (typeof m != 'undefined' && m) {if(m){}}`. And to verify that a variable contains a reference to function, BikiniProxy rewrites the Javascript code as follows `if (typeof func === 'function') {func()}`.

Algorithm 2 presents the algorithm of Line Skipper. First Line Skipper extracts the line, column, and URL of the resource from the failure point of the error. Line Skipper verifies that the current request is the resource that contains the error. Then Line Skipper extracts the AST from the Javascript code and looks for the element that is not defined. durieux2017productionFinally, the AST is transformed back to a textual form to be sent back to the client.

Object Creator. One of the most frequent error types at runtime is null dereference [3], and this also occurs much in Javascript [25]. Since Javascript is an untyped language, all null dereferences happening when setting properties can be handled with a generic empty object. For example, the code `var m = null; m.test = '';` will trigger the error `Cannot set property test of null` and can be avoided by adding `if (m == null) {m = {};}` before setting `m.test`.

The strategy of Object Creator is to initialize all null variables with a generic Javascript object before setting a property. When the execution continues, all the properties set in the generic object are readable later in the execution.

3.1.3 Monitoring and Self-healing Backend. The Monitoring and Self-healing Backend fulfills three tasks. The first task is to receive and store all the Javascript errors happening on client browsers. The Backend provides an API for BikiniProxy to query if a specific resource (URL) contains known errors.

The second task of the backend is to monitor the effectiveness of the different self-healing strategies. Each time that the section of the Javascript code rewrite by one of the five self-healing strategies is executed, an event is sent to BikiniProxy backend to keep track of the activation of the different strategies. Based on the number of activation and the number of errors per page, we can estimate the effectiveness of the self-healing strategies.

The third task is to provide a layout of communication with the developers about the monitored errors and the effectiveness of all self-healing strategies. For example, the following message can be given to the developer: "The strategy `Library Injector` has injected jQuery 22 times in the page `gallery.html` to handle the error `jQuery is not defined` ". This is valuable information to assist the developers in designing a permanent fix. And BikiniProxy backend also provides a visual interface that lists all the errors that the end-users face during the browsing of the web page.

3.2 Implementation

In this section, we present the prototype implementation of BikiniProxy. The source code and the usage examples are publicly available in [1]. The implementation of BikiniProxy's proxy is made in Javascript, based on a popular open-source proxy, anyproxy, by Alibaba.[3] The self-healing strategies based on HTML rewriting use htmlparser2[4]. The self-healing strategies based on rewriting the Javascript abstract syntax tree (AST) use the library babel.js[5].

3.3 Applicability

The usage of BikiniProxy is practically zero cost, and as such, it is widely applicable. First, it requires no change to the original web pages or applications. Second, the usage of an HTTP proxy in web application is very common. A BikiniProxy self-healing proxies can be set up by: 1) a company in front of their web content; 2) a SaaS-based provider 3) a hosting service. Thus, BikiniProxy is an highly-applicable solution.

BikiniProxy can be a new technology to enrich an existing proxy solution. For example, Cloudflare by Amazon [6] is a professional proxy provider developed by Amazon.com. It provides different services for websites: it offers caching, protection against DDoS (Attack by Denied-of-Service), etc. BikiniProxy's approach would perfectly integrate into Cloudflare (or another proxy company) to provide further added value.

4 EVALUATION

In our evaluation, we answer the following research questions.

RQ1. [Effectiveness] How is BikiniProxy effective at automatically fixing Javascript errors in production, without any user or developer involvement? The first research question studies if it is possible to handle field Javascript errors with our proxy based approach. We will answer this question by showing how real-world errors have been handled with one of our implemented self-healing strategies.

RQ2. [Outcome] What is the outcome of BikiniProxy's self-healing strategies on the page beyond making the error disappear? In this research question, we explore what are the possible outcomes of BikiniProxy on buggy web pages. We will answer this research question by presenting real-world case studies of different possible outcomes.

RQ3. [Comparison] Do the different self-healing strategies perform equivalently? Finally, we present to which extent the different self-healing strategies are used: Which type of errors are handled by the five self-healing strategies?

4.1 Protocol

We set up the following experimentation protocol to evaluate BikiniProxy. Our idea is to compare the behavior of an erroneous web page, against the behavior of the same, but self-healed page using our BikiniProxy. The comparison is made at the level of "web trace", a concept we introduce in this paper, defined as follows.

Definition 4.1 (web trace). A web trace is the loading sequence and rendering result of a web page. A web trace contains 1) the URL of the page 2) all the resources (URL, content, see Definition 3.1) 3) all the Javascript errors that are triggered when executing the Javascript resources and 4) a screenshot of the page at the end of loading.

Given a benchmark of web pages with Javascript errors, the following steps are made. The first step is to collect the web trace of each erroneous web page.

The second step is to collect the new web trace of each erroneous web page with BikiniProxy (Recall that all resources are rewritten by BikiniProxy self-healing strategies). In addition to the web trace, we also collect data about the self-healing process: the strategies that have been activated, defined by the tuple (initial error, strategy type).

The third step is to compare for each web page the original web trace against the self-healed web trace. The goal of the comparison is to identify whether BikiniProxy was able to heal the Javascript errors. For instance, the comparison may yield that all errors have disappeared, that is a full self-healing.

4.2 Construction of a Benchmark of Javascript Field Errors

To evaluate BikiniProxy, we need real-world Javascript that are reproducible. For each reproducible errors, we want to compare the behavior of the web page with and without BikiniProxy. To our knowledge, there is no publicly available benchmark of reproducible Javascript errors. Consequently, we create a new benchmark, we call it the DeadClick benchmark. The creation of our benchmark is composed of the following steps:

(1) randomly browse the web to discover web pages on Internet that have errors (see Section 4.2.1)
(2) collect the errors and their execution traces (see Section 4.2.2)
(3) ensure that one is able to reproduce the errors in a closed environment(see Section 4.2.3)

4.2.1 Web Page Finder. The first step of the creation of DeadClick is finding web pages that contain errors. In order to have a representative picture of errors on the Internet, we use a random

[3] anyproxy Github repository: https://github.com/Alibaba/anyproxy
[4] htmlparser2 Github repository: https://github.com/fb55/htmlparser2
[5] babel.js Github repository: https://github.com/babel/babel
[6] https://cloudflare.com

Table 1: Descriptive Statistics of DeadClick

Crawling stats	Value
# Visited Pages	96174
# Pages with Error	4282 (4.5%)
Benchmarks stats	**Value**
# Pages with Reproduced Errors	555
# Domains	466
# Average # resources per page	102.55
# Average scripts per page	35.51
# Min errors per page	1
# Average errors per page	1.49
# Max errors per page	10
# Average pages size	1.98mb

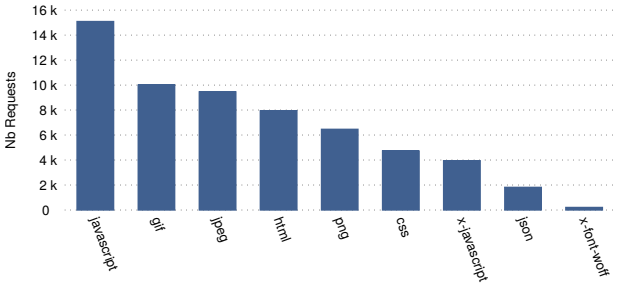


Figure 2: Bar plot of the number of requests by content-type.

approach. Our methodology is to take randomly two words from the English dictionary, and to combine those two words in a Google search request. A fake crawler then opens the first link that Google provides. If an error is detected on this page, the page URL is kept as tentative for the next step. The pros and cons of this methodology are discussed in Section 5

4.2.2 Web Trace Collector. The Javascript environment is highly dynamic and asynchronous. It means that many errors are transient and as such are not reproducible in the future, even in a very short period of time after their observation.

For identifying reproducing errors, our idea is to collect the web trace of the erroneous page and to try to reproduce the exact same web trace in a controlled environment, see Section 4.2.3

We implement the trace collection using the library puppeteer from Google[7], which provides an API to control Chrome in a programmatic manner. The big advantage of this library is that it uses the same browser engine as Chrome end-users, meaning that, by construction, DeadClick is only composed of errors that really happen on user browsers.

Since Javascript is mostly asynchronous, the Web Trace Collector waits for the end of loading where loading is defined as follows: 1) it opens the URL, 2) it waits for seven seconds, in order to load and execute all resources, in particular, Javascript files. 3) it scrolls the page to the bottom, in order to trigger additional initialization and Javascript execution. 4) it waits again for one second.

[7]puppeteer repository https://github.com/google/puppeteer

During this process, the Web Trace Collector logs 1) all errors that occur in the browser console and 2) all the requests (including the HTTP headers and the body) made from the browser. When the page is completely loaded, a screenshot of the page is taken, it provides a visual representation of the page. At the end of this process, for each page, the collected data is stored on disk if at least one error has been logged during the page browsing.

4.2.3 Web Page Reproduction. The last step of the benchmark creation consists of verifying that the collected errors can be reproduced. We consider that we succeed to reproduce the behavior of the web page when the observed errors during reproduction are identical to the ones in the originally collected web trace.

The reproduction of the error is done by browsing the erroneous page again but instead of using the resources from Internet, the Web Page Reproduction is cut from the Internet and only serves the resources stored on disk. In addition, it denies all the requests that have not been observed during the initial collection of the page.

4.3 Description of DeadClick

Table 1 gives the main statistics of DeadClick. The Web Page Finder visited a total of 96174 pages, and 4282 of the pages contains at least one error (4.5%), out of which 555 errors have been successfully reproduced. The final dataset contains errors from 466 different URL domains representing a large diversity of websites. There is on average 1.49 error per page, and each page has between one and ten errors.

Table 2 presents the top 10 of the errors present in DeadClick. In total DeadClick contains 53 different error types for a total of 826 collected errors. 69% of the Javascript errors are the first three error types: `XXX is not defined`, `Cannot read property XXX of null` and `XXX is not a function`.

Figure 2 presents the number of requests for the top 9 resource types. The external resources are mostly Javascript files. The rest of the distribution illustrates how complex modern web pages are. For sake open of open-science, DeadClick and its mining framework are available on Github [1].

4.4 RQ1: Effectiveness of Self-healing Web Applications

We now present the results of this first research question. Table 2 shows the top 10 types of errors in the considered benchmark and how they are handled by BikiniProxy. The first column contains the rank of the error type. The second column contains the error type, represented by the message of the error. The third column shows the number of web pages that contain this errors. The fourth column presents the number of different domains where the error is present. The fifth column contains the initial number of occurrences of the error in the DeadClick. The sixth column contains the number of healed error with BikiniProxy. The seventh column contains the percentage of the evolution, a negative percentage represents a decrease of the number of errors, a positive percentage an increase in the number of errors.

The first major result lies in the first row. It presents the error "XXX is not defined", which is the most common on the web according to our sampling. This error is present in 200 web pages across 166 different domains. It is thrown 307 times (meaning that some

Table 2: The Top 10 Error Types in DeadClick (left-hand side). The effectiveness of BikiniProxy (right-hand side).

#	Error messages	# Web Pages	# Domains	# Initial Errors	#Healed Errors with BikiniProxy	Improvement
1	XXX is not defined	200	166	307	184	59.93%
2	Cannot read property XXX of null	156	126	176	42	23.86%
3	XXX is not a function	92	86	111	11	9.9%
4	Unexpected token X	54	51	61	2	3.27%
5	Cannot set property XXX of null	21	17	24	11	45.83%
6	Invalid or unexpected token	18	12	21	0	0%
7	Unexpected identifier	13	11	15	0	0%
8	Script error for: XXX	8	3	10	2	20%
9	The manifest specifies content that cannot be displayed on this browser/platform.	5	5	7	0	0%
10	adsbygoogle.push() error: No slot	4	4	7	0	0%
	53 different errors	555	466	826	248	30.02%

Table 3: Analysis of the healing effectiveness per page.

Metric Name	# Pages	Percent
All Errors Disappeared	176/555	31.76%
Some Errors Disappeared	42/555	7.58%
Different/Additional Errors	140/555	25.27%
No Strategy Applied	196/555	35.31%

web pages throw it several times). With BikiniProxy, this error is healed 184/307 times, which represents a major improvement of 59.93%.

A second major result is that BikiniProxy is able to handle at least one error for the five most frequent Javascript errors. It succeeds to heal between 3.27% and 59.93% of the five most frequent Javascript errors in our benchmark. Overall, 248 errors are automatically healed, meaning that BikiniProxy reduces by 30.02% the number of errors in the benchmark;

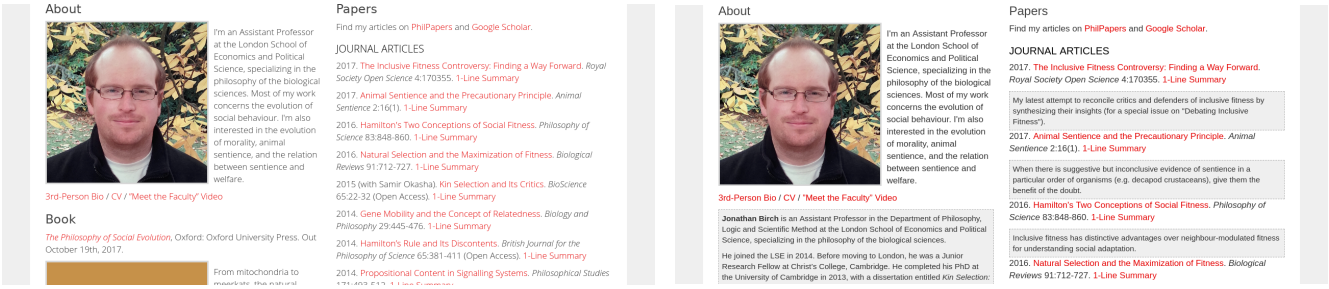
Now we discuss the categories of healed errors. We identify whether:

(1) *All errors disappeared:* no error happens anymore in the page loaded with BikiniProxy, meaning that one or a combination of rewriting strategies have removed the errors.
(2) *Some errors disappear:*, there are fewer errors than in the original web trace.
(3) *Different errors appear:* at least one error still, and it is a new error (new error type or new error location) that has never been seen before.
(4) *No strategy applied:* the error type is not handled by any of the strategies and thus there are the same errors than in the original web trace.

Table 3 presents the number of web pages per category. The first line of Table 3 shows that the number of web pages that have all the Javascript errors healed by BikiniProxy: BikiniProxy is able to handle all errors for 176/555 (31.76%) of the DeadClick benchmark. The second line shows the number of web pages that have been partially self-healed, by partially, we mean that the number of Javascript errors decrease with BikiniProxy but are still not zero: with BikiniProxy, 42 web pages contain fewer errors than before. The third line shows the number of web pages that have new errors than before: in 140/555 (25.27%) of the faulty pages, BikiniProxy is able to handle some errors but the rest of the execution produce new errors. The last line shows the number of web pages where none of the strategies has been applied: in 196/555 (35.13%) the errors are of a type that is not considered by BikiniProxy. To better understand the case where no strategy can be applied, we perform a manual qualitative analysis.

4.4.1 Case study: Unhandled Errors. The errors are unhandled when none of the five BikiniProxy rewriting strategies succeed to heal the errors. In our experiment, 196/555 of web pages have errors not healed, which represents 35.13% of the erroneous web pages of DeadClick. The first cause of non-healed errors is that the error type is not supported. For example, the web page http://dnd.wizards.com/articles/unearthed-arcana/artificer is loading a JSON file. However, the JSON file is invalid and the browser does not succeed to parse it which produces an `Unexpected token <` error. None of the five strategies is able to handle malformed JSON errors. The second cause of non-healed errors is that the self-healing strategies has not enough information to rewrite the resource. For example, the web page http://moreas.blog.lemonde.fr/2007/02/28/le-pistolet-sig-sauer-est-il-adapte-a-la-police/ contains the error `Cannot read property 'parents' of undefined`, this error should be healed with "Object Creator" rewriting. However, the trace of the error does not contain the URL of the resource that triggers this error, because the Javascript code has been unloaded. Consequently, "Object Creator" is not able to know which resource has to be rewritten to handle the error.

In summary, Table 3 shows that BikiniProxy is almost able to heal all the errors from a third of DeadClick. The second third of the benchmark is pages that cannot be healed with BikiniProxy. The last third contains web pages that are partially headed or that the self-healing strategies produce new errors.



(a) Without BikiniProxy, some content is missing

(b) Using BikiniProxy, the page loading is self-healed.

Figure 3: A real web page suffering from a Javascript bug. With BikiniProxy, the bug is automatically healed, resulting in additional information provided to the web page visitor. (if accepted, Dr Birch will be asked whether we keep the screenshot as is or whether we anonymize it)

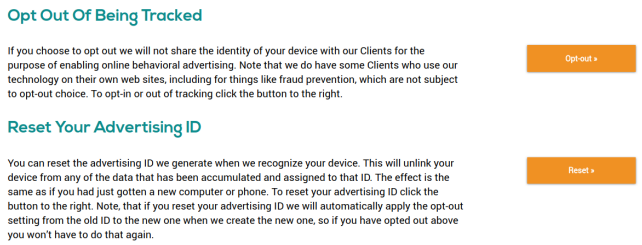


Figure 4: The two buttons in orange are missing in the original buggy page. When BikiniProxy is enabled, the two orange buttons provide the user with new user-interface features.

Answer to RQ1. The results of this research question shows that BikiniProxy is effective the handle the five most frequent Javascript errors present in our benchmark. With the currently implemented self-healing rewriting strategies, BikiniProxy is able to fully heal 248/826 (30.02%) of all errors, representing 196/555 (31.76%) of all buggy web pages of our benchmark.

4.5 RQ2: Outcome

In this second research question, we focus on category "All Errors Disappeared", and further refines the classification as follows:

(1) The errors have disappeared but no behavioral change can be seen by the end-user.
(2) The errors have disappeared and new UI features (eg. new buttons) are available to the end-user.
(3) The errors have disappeared and new content is available for the end-user.

Contrary to RQ1, it is not possible to automatically classify all pages with this refined category, because it requires human-based assessment of what is new content or new features. For this reason, we answer to this RQ with a qualitative case study analysis.

4.5.1 Case study: Error Handled but No Behavior Change. A healed error does not automatically result in a behavior change in the application. For example, this is the case of website https://cheapbotsdonequick.com/source/bethebot, which triggers the error `"module" is not defined`. This error is triggered by line `module.exports = tracery;`. This type of line is used to make a library usable by another file in a Node.js environment. However, Node.js has a different runtime from a browser, and the module object is not present, resulting in the error. With BikiniProxy, the self-healing strategy "Object Creator" automatically initializes the variable `module`, however, since this line is the last line of the executed Javascript file, this has absolutely no further consequence on the execution or the page rendering. This means that the error was irrelevant. However, from a self-healing perspective, this cannot be known in advance. From a self-healing engineering perspective, the takeaway is that it is more simple to heal irrelevant errors than to try to predict their severity in advance.

4.5.2 Case study: new Feature Available. One possible outcome of BikiniProxy is that the self-healing strategy unlocks new features. For example, this the case of https://bluecava.com/. This page has an error, shown in Listing 1, which is triggered because the developer directly accesses the content of Ajax requests without checking the status of the request. However, there are requests that are denied due to cross-domain access restrictions implemented in all browsers. Since the developer did not verify if there is an error before accessing the property 'id' on a null variable, the Javascript event loop crashes.

With BikiniProxy, the self-healing strategy "Object Creator" ensures the initialization of the variable if it is null. This execution modification allows the execution to continue and to finally enter in an error handling block written by the developer, meaning that the event loop does not crash anymore. The execution of the page continues and results in two buttons being displayed and enabled for the end-user. Figure 4 presents the two buttons that are now available for the user.

4.5.3 Case study: new content available. One other outcome of BikiniProxy is that additional content is displayed to the end-user.

Let us consider the web page http://personal.lse.ac.uk/birchj1/ that is the personal page of a researcher. This page triggers the following error: `$ is not defined at (index):20`

This error is thrown because a script in the HTML page calls the jQuery library before the library is loaded. The script that throws

Listing 1: Error on the web page https://bluecava.com/

```
Uncaught TypeError:
  Cannot read property 'id' of null
    at bluecava.js?v=1.6:284 ...
    at post (bluecava.js?v=1.6:40)
    at identify (bluecava.js?v=1.6:156) ...
```

Table 4: The number of activations of each self-healing strategy and the number of error types that the strategy can handle.

Self-healing Strategies	# Activations	# Supported Error Types
Line Skipper	233	4
Object Creator	109	2
Library Injector	75	3
HTTP/HTTPS Redirector	18	NA
HTML Element Creator	14	2

the error is responsible to change the visibility of some content in the page. Consequently, because of the error, this content stays hidden for all visitors of the page.

Using BikiniProxy, the error is detected as being caused by a missing jQuery library. This error is healed by rewriting strategy "Library Injector" Consequently, BikiniProxy rewrites the content of the page by injecting the jQuery library and sends back the rewritten page to the browser. When the rewritten script is executed, jQuery is available, and consequently, the script is able to change the visibility of hidden HTML elements, resulting in newly visible content.

Figure 3 presents the visual difference between the page without BikiniProxy (left side), and loaded with BikiniProxy (right side). All the elements in a grey box on the right-hand side are missing on the left image, they have appeared thanks to self-healing.

Answer to RQ2. All three possible outcomes: (1) no visible change; (2) new features; and (3) new content; are present in our benchmark. The fact that BikiniProxy is able to automatically restore broken features or broken content represents a significant improvement to the web experience of all users in front of a BikiniProxy instance.

4.6 RQ3: Strategies

In this research question, we compare the five different self-healing strategies of BikiniProxy. For each strategy, Table 4 shows the number of times it has been activated to heal errors of DeadClick, and the last column presents the number of different error types for which the strategy has been selected. For example, the first row of Table 4 shows that "Line Skipper" has been selected to handle 233 errors, and it has healed four different error types.

The most used strategy is "Line Skipper" with 233 activations. It is also the strategy that supports the highest number of different error type: 1) "XXX is not defined", 2) "XXX is not a function", 3) "Cannot read property XXX of null", 4) "Cannot set property XXX of null". The second most used strategy is "Object Creator" with 109 errors for which it has initialized a null variable. This strategy handles two different error types: "Cannot set property XXX of null" and "Cannot read property XXX of null". These two strategies have something in common, they target the failure point, the symptom, and not the root cause (the root cause is actually unknown). For example, the error `CitedRefBlocks is not defined` is triggered in the web page https://www.ncbi.nlm.nih.gov/pmc/articles/PMC504719/, because the function `CitedRefBlocks` is not defined. Line Skipper strategy avoids the error by skipping the method call, it is a typical example of a fix at the failure point and not at the root cause of the absence of `CitedRefBlocks`.

On the contrary, "Library Injector" addresses the root cause of the problem: the missing library is extracted from the error message and it is used to rewrite the content of the request. In this case, BikiniProxy exploits the fact that we have a direct relation between root cause (no included library) and symptom (unknown used library name) for this error type.

The case of "HTTP/HTTPS Redirector" is the opposite. Recall that "HTTP/HTTPS Redirector" directly looks in the HTML body of the resource if there are scripts that will be blocked. This means that the rewriting addresses the root cause of potential future problems. For example, the page https://corporate.parrot.com/en/documents tries to load the resource http://www.google-analytics.com/urchin.js, but the request is blocked by the browser (HTTP request in an HTTPS page). Consequently, the Google tracking library is not loaded and function `urchinTracker` is not defined, resulting in the error `urchinTracker is not defined`. "HTTP/HTTPS Redirector" strategy rewrites the URL of the resource in the <SCRIPT>tag to https://www.google-analytics.com/urchin.js, and this fixes the error of the page. This strategy can potentially fix error types that we cannot envision, hence, we do not know the exact number of handled error types, so we put "NA" in Table 4.

Finally, strategy "HTML Element Creator" is applied to more rare errors happening only 14 times in our benchmark.

Answer to RQ3. The usage of strategies depends on the prevalence of the underlying errors in the field: in our experiment, the most used strategy is "Line Skipper" because it covers four common error types.

5 THREAT TO VALIDITY

We now discuss the threats to the validity of our experiment. First, let us discuss internal validity. Our experiment is relying on the implementation of our prototype, consequently, a bug in our code may threaten the validity of our results. However, since the source code of the approach and of the benchmark is publicly available [1], future researchers will be able to identify these potential bugs.

Second, a threat to the external validity relates to the representativeness of the benchmark. An open question is whether our sampling of the Web is representative. First, recall that by construction, there is no cherry-picking because the page selection is fully random. However, the remaining threats are: 1) our sampling is done over the English web 2) our sampling is impacted by Google (recall that we select the first result). 3) whether 96174 visited pages

is enough compared to the trillions of pages of the Internet. To our knowledge, there is no work on the representativity of a web sample for Javascript bugs, and as such no definitive answer to this question.

Our approach has been carefully designed to maximize representativity, with two main advantages: 1) the randomness of keyword choice allows us to discover website about many different topics, done by a variety of persons, with different backgrounds (on website on CSS done by a web developer is likely to have fewer errors then a website on banana culture done by a hobbyist). 2) the ranking of Google provides us with a filter which favors popular websites. If errors are detected on those websites, they likely affect many users. It means that if BikiniProxy heals those errors, it would have a large impact.

6 RELATED WORK

Javascript Error. Several studies on client-side Javascript been have been made by Ocariza et al [27, 24, 25]. In 2011, they have shown that most websites contain Javascript errors even in the top 100 of Alexa [27]. They have also investigated [24, 25] the nature of the Javascript errors as follows. They manually analyze Javascript bug reports from various web applications and Javascript libraries. They find that the majority of reported Javascript bugs are related to the Document Object Model (DOM). None of this work has explored self-healing strategies for the web.

Now, we discuss the works on reproducing Javascript errors or to extract regression tests. Wang et al. [36] present a technique to reproduce sequence of events that leads to a Javascript error. Schur et al. [34] present a fully automatic tool to generate test scripts based on the behavior of multi-user web applications. The goal of those works is different ours: the focuses on creating tests while we focus on healing the error on the fly, in production.

Hanam et al. [17] present BugAID a data mining technique for discovering common unknown bug patterns in server-side Javascript. Hanam et al. focus on server-side bugs, on the contrary, BikiniProxy targets client-side Javascript code and it heals the errors on the fly.

Javascript Program Repair. There have been several repair tools targeting Javascript front-end code. Ocariza et al. [26] present Vejovis, a technique that suggests Javascript code modifications to handle DOM-related errors. Pradel et al. [30] and Bae et al. [4] proposed tools for detecting type inconsistencies and web API misuses in Javascript respectively. They also present common fault types and common web API misuse patterns. Roy et al. [33] present X-PERT, an automatic technique to detect cross-browser issues in web applications. Those works are offline program repair, producing suggestions for developers, while BikiniProxy is online self-healing, in production, without the developer is the loop.

Carzaniga et al. [11] propose a technique that automatically applies workarounds to handle API issues. The workarounds are based on a set of manually written API-specific alternative rule. The manual step makes the approach not fully automated. On the contrary, our approach does not require a manual database of workarounds to be set up. In subsequent work, the same group has proposed a way to automatically mine those workarounds [12]. However, those workarounds do not consider generic Javascript errors as we do, they target API specific errors for which workarounds have been identified or mined.

Proxy-based Approaches. Several approaches use a proxy-based architecture in their contribution. Kiciman et al. [18] present a web proxy named AjaxScope, that instruments the Javascript code to monitor the performance of web applications. It does monitoring and not self-healing. Zhang et al. [37] present a technique to change the user interface of mobile applications on the fly. In particular, they aim at providing accessible UIs to blind users.

Self-healing in Production. Rx [31] is a self-healing system that changes the environment upon failures. Rx employs checkpoint-and-rollback for re-executing the buggy code when failures happen. Assure [35] is a self-healing system also based on checkpointing. Perkins et al. [28] propose ClearView, a system for automatically repairing errors in production by monitoring low-level registers to learn invariants. When ClearView detects that an invariant is violated, it forces the restoration of the system to a previous valid state. Rinard et al. [32] present a technique called "failure-oblivious computing" to avoid illegal memory accesses by wrapping each memory operation during the compilation process. Long et al. [21] also explore this idea with the concept of "recovery shepherding".

All those techniques target low-level code, typically C code, which is significantly different from Javascript and a browser runtime. BikiniProxy is a self-healing for a different software stack, and its five rewriting strategies can be considered as novel failure-oblivious computing models for the web. Gu et al. [15] present Ares, a runtime error recovery technique for Java exceptions using JavaPathFinder (JPF). Those contributions target Java program and modify the source code of the application in production.

Carzaniga et al. [9, 10] present ARMOR a technique for self-healing Java applications based on workaround that can be deployed at runtime. Armor injects checkpoint and rollback mechanism in the application, this mechanism is used to rollback when a failure is detected and explore workaround alternatives.

7 CONCLUSION

In this paper, we have presented BikiniProxy, a novel technique to provide self-heal capabilities for client-side Javascript errors. This technique can be used without modifying the code or setup of web applications. The concept of self-healing proxy that we propose in this paper is novel, and complements network, minimization, optimization and security proxies. We have evaluated our technique on 555 web pages with Javascript errors, randomly collected on the Web. Our qualitative and quantitative evaluation has shown that BikiniProxy is able to make all Javascript errors disappear and to provide the user with new features and content. Future work is required to devise new self-healing rewriting strategies for solving the maximum number of Javascript runtime errors.

REFERENCES

[1] Bikiniproxy repository. https://github.com/Spirals-Team/bikiniproxy/, 2018.
[2] Uage ranking of javascript libraries. https://w3techs.com/technologies/overview/Javascript_library/all, 2018.
[3] We crunched 1 billion java logged errors - here's what causes 97% of them. https://blog.takipi.com/we-crunched-1-billion-java-logged-errors-heres-what-causes-97-of-them/, 2018.

[4] S. Bae, H. Cho, I. Lim, and S. Ryu. Safewapi: Web api misuse detector for web applications. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 507–517. ACM, 2014.

[5] B. Baudry and M. Monperrus. The multiple facets of software diversity: Recent developments in year 2000 and beyond. *ACM Computing Surveys (CSUR)*, 48(1):16, 2015.

[6] E. Berger. Software Needs Seatbelts and Airbags. *ACM Queue*, 10, 2012.

[7] M. K. Bowman-Amuah. Load balancer in environment services patterns, June 10 2003. US Patent 6,578,068.

[8] G. Candea, E. Kiciman, S. Zhang, P. Keyani, and A. Fox. Jagr: An autonomous self-recovering application server. In *Autonomic Computing Workshop. 2003. Proceedings of the*, pages 168–177. IEEE, 2003.

[9] A. Carzaniga, A. Gorla, A. Mattavelli, and N. Perino. A self-healing technique for java applications. In *Software Engineering (ICSE), 2012 34th International Conference on*, pages 1445–1446. IEEE, 2012.

[10] A. Carzaniga, A. Gorla, A. Mattavelli, N. Perino, and M. Pezze. Automatic recovery from runtime failures. In *Proceedings of the 2013 International Conference on Software Engineering*, pages 782–791. IEEE Press, 2013.

[11] A. Carzaniga, A. Gorla, N. Perino, and M. Pezzè. Automatic workarounds for web applications. In *Proceedings of the eighteenth ACM SIGSOFT international symposium on Foundations of software engineering*, pages 237–246. ACM, 2010.

[12] A. Carzaniga, A. Gorla, N. Perino, and M. Pezze. Automatic workarounds: Exploiting the intrinsic redundancy of web applications. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 24(3):16, 2015.

[13] B. Demsky and M. Rinard. Automatic detection and repair of errors in data structures. *ACM SIGPLAN Notices*, 38(11):78–95, 2003.

[14] T. Durieux, B. Cornu, L. Seinturier, and M. Monperrus. Dynamic patch generation for null pointer exceptions using metaprogramming. In *Software Analysis, Evolution and Reengineering (SANER), 2017 IEEE 24th International Conference on*, pages 349–358. IEEE, 2017.

[15] T. Gu, C. Sun, X. Ma, J. Lü, and Z. Su. Automatic runtime recovery via error handler synthesis. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering*, pages 684–695. ACM, 2016.

[16] S. A. Gurguis and A. Zeid. Towards autonomic web services: Achieving self-healing using web services. In *ACM SIGSOFT Software Engineering Notes*, volume 30, pages 1–5. ACM, 2005.

[17] Q. Hanam, F. S. d. M. Brito, and A. Mesbah. Discovering bug patterns in javascript. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 144–156. ACM, 2016.

[18] E. Kiciman and B. Livshits. Ajaxscope: a platform for remotely monitoring the client-side behavior of web 2.0 applications. In *ACM SIGOPS Operating Systems Review*, volume 41, pages 17–30. ACM, 2007.

[19] T. Krueger, C. Gehl, K. Rieck, and P. Laskov. Tokdoc: A self-healing web application firewall. In *Proceedings of the 2010 ACM Symposium on Applied Computing*, pages 1846–1853. ACM, 2010.

[20] C. Lewis and J. Whitehead. Runtime repair of software faults using event-driven monitoring. In *Software Engineering, 2010 ACM/IEEE 32nd International Conference on*, volume 2, pages 275–280. IEEE, 2010.

[21] F. Long, S. Sidiroglou-Douskos, and M. Rinard. Automatic runtime error repair and containment via recovery shepherding. In *ACM SIGPLAN Notices*, volume 49, pages 227–238. ACM, 2014.

[22] A. Luotonen. *Web proxy servers*. Prentice-Hall, Inc., 1998.

[23] M. Monperrus. A critical review of automatic patch generation learned from human-written patches: essay on the problem statement and the evaluation of automatic software repair. In *Proceedings of the 36th International Conference on Software Engineering*, pages 234–242. ACM, 2014.

[24] F. Ocariza, K. Bajaj, K. Pattabiraman, and A. Mesbah. An empirical study of client-side javascript bugs. In *Empirical Software Engineering and Measurement, 2013 ACM/IEEE International Symposium on*, pages 55–64. IEEE, 2013.

[25] F. S. Ocariza, K. Bajaj, K. Pattabiraman, and A. Mesbah. A study of causes and consequences of client-side javascript bugs. *IEEE Transactions on Software Engineering*, 43(2):128–144, 2017.

[26] F. S. Ocariza Jr, K. Pattabiraman, and A. Mesbah. Vejovis: suggesting fixes for javascript faults. In *Proceedings of the 36th International Conference on Software Engineering*, pages 837–847. ACM, 2014.

[27] F. S. Ocariza Jr, K. Pattabiraman, and B. Zorn. Javascript errors in the wild: An empirical study. In *Software Reliability Engineering (ISSRE), 2011 IEEE 22nd International Symposium on*, pages 100–109. IEEE, 2011.

[28] J. H. Perkins, S. Kim, S. Larsen, S. Amarasinghe, J. Bachrach, M. Carbin, C. Pacheco, F. Sherwood, S. Sidiroglou, G. Sullivan, W.-F. Wong, Y. Zibin, M. D. Ernst, and M. Rinard. Automatically patching errors in deployed software. In *Proceedings of the ACM SIGOPS 22Nd Symposium on Operating Systems Principles*, SOSP '09, pages 87–102, New York, NY, USA, 2009. ACM.

[29] J. C. Pistriotto and K. Montinola. Method and apparatus for configuring a client to redirect requests to a caching proxy server based on a category id with the request, Oct. 24 2000. US Patent 6,138,162.

[30] M. Pradel, P. Schuh, and K. Sen. Typedevil: Dynamic type inconsistency analysis for javascript. In *Proceedings of the 37th International Conference on Software Engineering-Volume 1*, pages 314–324. IEEE Press, 2015.

[31] F. Qin, J. Tucek, J. Sundaresan, and Y. Zhou. Rx: Treating Bugs as Allergies—A Safe Method To Survive Software Failures. SIGOPS OSR '05.

[32] M. C. Rinard, C. Cadar, D. Dumitran, D. M. Roy, T. Leu, and W. S. Beebee. Enhancing server availability and security through failure-oblivious computing. In *OSDI*, volume 4, pages 21–21, 2004.

[33] S. Roy Choudhary, M. R. Prasad, and A. Orso. X-pert: accurate identification of cross-browser issues in web applications. In *Proceedings of the 2013 International Conference on Software Engineering*, pages 702–711. IEEE Press, 2013.

[34] M. Schur, A. Roth, and A. Zeller. Procrawl: Mining test models from multi-user web applications. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis*, pages 413–416. ACM, 2014.

[35] S. Sidiroglou, O. Laadan, C. Perez, N. Viennot, J. Nieh, and A. Keromytis. Assure: automatic software self-healing using rescue points. In *ACM Sigplan Notices*, volume 44, pages 37–48. ACM, 2009.

[36] J. Wang, W. Dou, C. Gao, and J. Wei. Jstrace: Fast reproducing web application errors. *Journal of Systems and Software*, 2017.

[37] X. Zhang, A. S. Ross, A. Caspi, J. Fogarty, and J. O. Wobbrock. Interaction proxies for runtime repair and enhancement of mobile application accessibility. In *CHI*, 2017.